Reconstructing Global Models from Federated Client Weights Using Neural Networks

Arsh Javed | Ashwini Kumar Singh | Arun Chauhan

Department of Computer Science and Engineering

Graphic Era Deemed to be University DEHRADUN

Motivation

Federated learning keeps raw data local, but the server still has to combine models shaped by different client cohorts.

Problem: Aggregating models trained on non-IID data.

Our solution: Apply neural networks to predict aggregate functions.

INTRODUCTION

Can the aggregation function itself be learned from data?

The introduction surveys stronger aggregation strategies, but the core question remains open.

FedAvg / FedMedian

Heuristic mean and coordinate-wise median aggregation.

SCAFFOLD / FedLAW

Variance reduction and learned aggregation weights.

FedMA / FedMR / FedLPA / pFedClub

Matching, recombination, layer-wise, and personalized aggregation.

Open question in this paper

Can the aggregation function itself be learned from data rather than fixed by a heuristic?

This paper

Reconstruction turns aggregation into a supervised prediction problem with layer-specific neural networks.

METHODOLOGY

Problem Formulation

Aggregation target

$$W_g = \mathrm{A}(W_1, W_2, W_3, W_4)$$

Learned layer mapping

$$\hat{w}_g^{(l)} = f_{\theta_l}(w_1^{(l)}, w_2^{(l)}, w_3^{(l)}, w_4^{(l)})$$

The paper learns one predictor per layer and trains it to minimize squared error to the centralized target weights.

What the paper tests

Do client weights contain enough signal to reconstruct the centralized global weights?

Step 1

Collect aligned parameter values from the four client models.

Step 2

Predict the centralized target weight with a layer-specific MLP.

Step 3

Repeat across layers and assemble the reconstructed global model.

This keeps the learning target concrete: every predictor estimates only the global value for one layer.

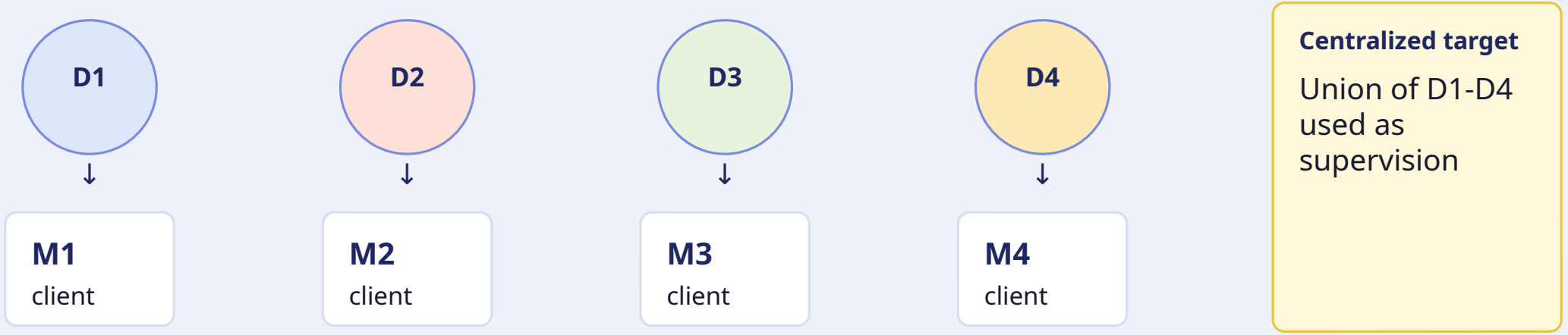
METHODOLOGY

Ground Truth Generation

The paper supervises reconstruction with a centralized model trained on the union of the four client datasets.

Source corpus split into four distinct datasets

The same preparation step defines D1, D2, D3, and D4 for the client models and for the centralized target.



Local client models and the centralized target are trained from aligned dataset splits.

A Layer-wise and Adaptive Framework

Each parameter tensor becomes its own prediction task, and layer size determines the prediction-network settings.

Prediction network for one layer

Input
4 client values for the same parameter

→

Hidden block
Linear
BatchNorm1d
GELU
Dropout

→

Output
1 target weight

Why adaptation matters

The paper uses size-aware hidden dimensions, learning rates, dropout, and weight decay because parameter counts vary sharply across layers.

Small layers (< 412,876 params)

lr 5×10^{-5}
hidden [64, 32]
dropout 0.5

Medium layers

lr 1×10^{-4}
hidden [128, 64, 32]
dropout 0.3

Large layers (> 1,651,507 params)

lr 2×10^{-4}
hidden [256, 128, 64]
dropout 0.15

Each prediction network is trained for 20 epochs with CosineAnnealingLR.

METHODOLOGY

Training the Prediction Networks

One layer at a time, the method turns aggregation into a supervised prediction pipeline.

1. Stack inputs

For each layer, collect the same parameter value from M1 through M4.

→

2. Split 70/30

Train on 70 percent of parameters and hold out 30 percent for reconstruction.

→

3. Train layer MLP

Apply RobustScaler, then optimize the layer-specific MLP with MSE and AdamW.

→

4. Merge outputs

Merge observed and predicted weights to form the final layer.

Optimization details

20 epochs, MSE loss, AdamW, and a CosineAnnealingLR schedule for each prediction network.

Stability and efficiency

Mixed precision and gradient clipping support training; the key output is the reconstructed 30 percent of weights.

Experimental Setup

4
client models and non-overlapping datasets

5
epochs for each client model

128
maximum sequence length

M

Case study

mental-bert-base-uncased on four non-overlapping client splits for binary depression classification.

T

Client training

AdamW, batch size 32, FP16 mixed precision, early stopping.

P

Preprocessing

Lowercase, remove URLs and mentions, split 70/15/15, max length 128.

B

Baselines and metrics

FedAvg and coordinate-wise FedMedian; evaluate Accuracy, MSE, and R-squared.

RESULTS

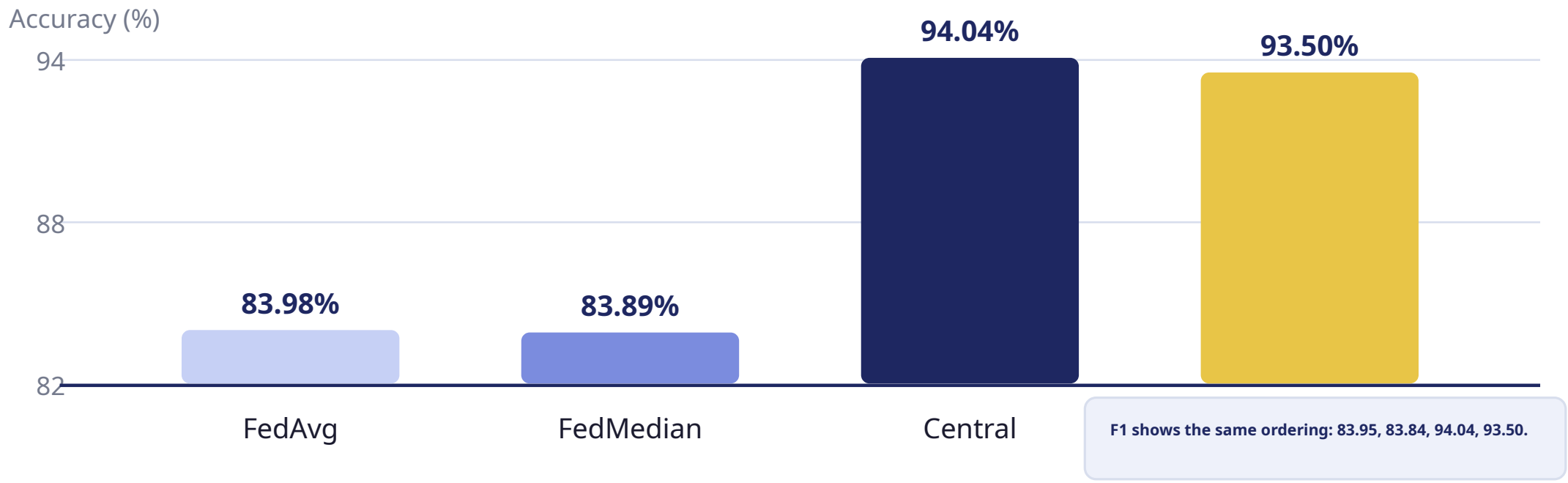
Performance Comparison of Aggregation Methods

The reconstructed model closes almost all of the gap to the centralized oracle and clearly beats both heuristic baselines.

93.50%
reconstructed accuracy

0.953
average reconstruction R-squared

99.4%
of centralized accuracy retained



RESULTS

Parameter Space Analysis

The reconstructed model looks slightly more regularized while staying near the client center.

2.14%
near-zero weights in M Final

0.9975
average cosine similarity to clients

0.965
FedAvg-target reconstruction R-squared

Central vs reconstructed global model

Metric	M Central	M Final
L1 norm	3.739 x 10^6	**3.708 x 10^6**
L2 norm	471.84	**468.04**
Sparsity	1.95%	**2.14%**

Weight structure and sparsity

The classifier layer shows a sharper peak at zero in M Final than in M Central.

Generalizability

The same framework also learns a FedAvg target, reaching R-squared 0.965.

Where it still struggles

Small layers with only 1,075 samples can produce negative R-squared values.

Thank You !

Conclusion

- Client weights carry enough signal for reconstruction.
- Learned aggregation nearly matches the centralized oracle.
- The method still needs an oracle target and full client weights.

Future work

- Cross-architecture generalization
- Communication-efficient architectures
- Privacy enhancements
- Improved small-layer prediction